

面向基础设施与运维体系的 AI 自动化运维框架设计

原始需求：自由发挥，设计 temporal 自动ai运维的机制，设计一个 ai devops框架；完成后fork到仓库发给我

说明：

因该Homework明显偏开发视角，偏离我目前的技术框架，在未明确工作之前，接入新的技术学习时间明显超过预期成本。

本人当前强项在基础架构、运维体系与项目落地。

故本方案主要从：

- 系统架构设计
- 运维可落地性
- 风险控制
- 稳定性保障

等角度进行一个简要设计。

未在本次作业中深入实现具体框架（如 Temporal API），而是强调系统层面的设计与项目实现思路。后续若实际落地，可根据团队技术栈补充具体实现。

1. 背景

随着系统规模的不断扩大（多服务、分布式、高并发），传统运维模式逐渐面临以下问题：

- 告警处理依赖人工经验，响应效率不稳定
- 故障定位耗时长，依赖个体能力
- 运维流程难以标准化与复用
- 自动化程度有限，缺乏统一调度与决策能力

与此同时，大模型（LLM）与 AI Agent 能力逐渐成熟，使其可以参与：

- 告警分析与归因
- 日志理解与总结
- 运维决策辅助
- 自动化操作建议生成

因此，有必要设计一套**AI参与但可控的运维自动化体系**。

2. 设计目标

本方案目标如下：

- 提升故障处理效率
- 标准化运维流程（自动化）
- 引入 AI 辅助分析能力
- 保证系统稳定性、安全性与可控性

- 支持多服务、多环境的自动化运维

3. 设计原则

1. AI辅助而非替代人工
2. 高风险操作必须可控（审批 / 限制）
3. 所有操作可审计
4. 支持回滚与失败恢复
5. 与现有运维体系兼容（Prometheus / Shell / Ansible 等）
6. 可扩展（支持替换 AI 模型与调度引擎）

4. 总体架构设计（运维视角）

本方案不从“技术实现组件”出发，而从**运维流程与系统职责拆分**进行设计。

核心思路：

将 AI 能力接入到现有运维流程中，而不是替代现有体系。

4.1 架构分层（运维职责拆分）

系统划分为五个核心层：

```
[事件输入层]
↓
[运维上下文构建层]
↓
[AI分析与决策层]
↓
[执行与调度层]
↓
[审计与可观测层]
```

4.2 各层职责说明

① 事件输入层

负责统一接收运维事件：

- 监控告警
- 日志系统异常
- 人工触发（运维/开发）
- 定时巡检任务
- 工单系统

特点：

- 不改造现有系统
- 只做统一入口

② 运维上下文构建层

这是整个系统**最关键的一层（运维核心价值）**将“告警”转为“可判断的问题上下文”：

输入示例：

- 主机信息
- 服务信息
- 最近变更记录
- 监控指标（CPU/IO/带宽等）
- 日志摘要

输出：

```
[结构化问题描述]
= 哪个服务
+ 哪台机器
+ 当前状态
+ 最近变化
+ 影响范围
```

本层本质：**把“人脑经验”结构化**

③ AI分析与决策层

职责：

- 告警归类
- 初步原因分析
- 生成处理建议（Plan）

输出示例：

```
可能原因：
- 流量突增
- 进程异常
- 资源不足

建议操作：
- 查看进程
- 限流
- 重启服务
```

关键原则：AI 只给建议，不直接做高风险执行

④ 执行与调度层

负责把“建议”变成“可执行动作”

执行方式：

- Shell 脚本

- Ansible
- 运维工具链
- Kubernetes / 云 API

调度方式（可选）：

- Temporal
- Airflow
- Argo
- 自研调度器

本层本质：**把经验流程自动化**

⑤ 审计与可观测层

记录所有行为：

- 谁触发
- AI给了什么建议
- 执行了什么
- 成功/失败
- 是否人工介入

用于：

- 复盘
- 风险追溯
- 优化模型

本层说明：**在高可用性运维工作中，这层是必须的。**

5. 核心流程设计

示例场景：CPU异常自动处理

阶段 1：事件触发

```
[监控告警触发]
↓
[发送 CPU 异常告警]
↓
[进入事件输入层]
```

阶段 2：上下文构建

[收集监控数据]

↓

[收集进程信息]

↓

[收集最近变更]

↓

[生成结构化上下文]

阶段 3：AI分析

[输入：上下文]

↓

[AI分析]

↓

[输出：原因 + 建议]

阶段 4：执行决策

[判断风险等级]

低风险：

→ 自动执行

中风险：

→ 人工确认后执行

高风险：

→ 仅提示

阶段 5：执行与记录

[执行脚本 / 操作]

↓

[记录执行结果]

↓

[更新系统状态]

阶段 6：审计与复盘

[记录完整流程]

↓

[统计处理时间]

↓

[优化策略]

6. 风险控制设计

6.1 操作分级机制

等级	类型	示例
L1	自动执行	清理日志
L2	审批执行	重启服务
L3	禁止自动	数据删除

6.2 执行限制

- 限制执行范围（单机 / 单服务）
- 限制执行频率
- 支持 dry-run
- 黑白名单机制

6.3 人工兜底机制

- AI 只作为辅助
- 所有关键操作必须人工可介入
- 支持中断流程

6.4 回滚机制

- 所有关键操作必须支持回滚
- 记录执行前状态

7. 与现有运维体系的融合

本方案**不推翻**现有体系，而是**叠加能力**

例如已有系统：

- 监控：例如 Prometheus
- 日志：例如 ELK / Loki
- 执行：例如 Shell / Ansible
- ...

AI系统作为：“**增强层**”

8. 关于 Temporal

Temporal 可作为工作流调度的一种实现方式，适用于长流程任务编排与状态管理。但本方案不依赖具体实现工具，可根据实际情况选择，如(以下工具搜集于网络)：

- Temporal
- Airflow
- Argo Workflows
- 或自研调度系统

重点在于流程设计，而非框架绑定。

9. 设计总结

本方案核心

不是：不是用AI替代运维

而是： **让AI成为运维体系中的辅助决策能力**

本方案强调：

- 稳定性优先
- 风险可控
- 流程可复用
- 系统可落地

AI的价值：

- 提升效率
- 减少人为错误
- 标准化经验

运维的价值：

- 控制风险
- 保证稳定
- 兜底系统

10.说在最后：

本设计更偏基础设施与运维体系视角，重点在“如何让AI能力在生产环境中可控落地”。相比纯开发型方案，本方案更关注：

- 稳定性
- 风险控制

- 运维流程标准化
 - 系统长期可维护性
-

在实际工作中开发与运维视角存在明显差异化，不同的工种不可能完全同质化管理，各司其职，专注完成各自擅长的工作，相互配合，才是团队最高效的工作方式。

该文档仅限于崔高磊面试使用